

COAL NOTES

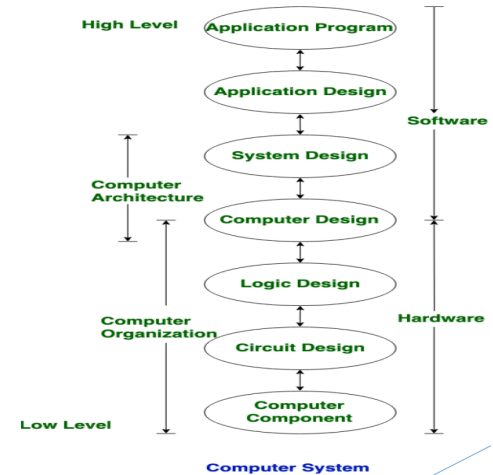
(compiled by Nabira Khan using course books, instructor slides, and online resources)

BASIC CONCEPTS

Computer Architecture: Computer architecture is the functional description of a computer's design and behavior, outlining what the system does before its physical organization (*describes what the computer does*)

Computer Organization: Computer organization defines how hardware components are structured and interconnected to implement the architectural specifications, focusing on how the system operates (*describes how it does it*)

Aspect	Computer Architecture	Computer Organization
Purpose	Helps us understand the functionalities of a system	Tells how all units are arranged and interconnected
Focus	Deals with high-level design issues	Deals with low-level design issues
Involves	Logic (Instruction sets, Addressing modes, Data types, Cache optimization)	Physical components (Circuit design, Adders, Signals, Peripherals)



Assembly Language:

- a low-level language designed for a specific type of processor
- uses short mnemonics such as ADD, SUB, and MOV
- has a one-to-one relationship with machine language
- every assembler has its own assembly language

Assembler: An assembler is a utility program that converts source code programs from assembly language into machine language

Linker: A linker is a utility program that combines individual files an assembler creates into a single executable program.

- The linker reads the object file and checks to see if the program contains any calls to procedures in a link library.
- The linker copies any required procedures from the link library, combines them with the object file, and produces the executable file.
- Microsoft 16-bit linker is LINK.EXE and 32-bit is Linker LINK32.EXE.

OS Loader: A program that loads executable files into memory, and branches the CPU to the program's starting address, (may initialize some registers (e.g. IP)) and the program begins to execute.

Listing File: A listing file includes the source code with line numbers, instruction addresses, machine code bytes in hexadecimal, and a symbol table with program identifiers and segments. Advanced programmers sometimes use the listing file to get detailed information about the program.

Debugger: lets you step through a program while it's running and examine registers and memory.

Machine Language: is a numeric language specifically understood by a computer's processor

(the CPU). All x86 processors understand a common machine language. A single machine-language instruction can take up one or more bytes of code.

High-Level Language: a programming language that enables a programmer to write programs that are more or less independent of a particular type of computer. High-level languages such as Python, C++, and Java have a one-to-many relationship with assembly language and machine language.

Registers: a high-speed storage location directly inside the CPU, designed to be accessed at a much higher speed than conventional memory, and holds immediate results of operations.

Device drivers: are programs that translate general operating system commands into specific references to hardware details.

ASSEMBLY LANGUAGE CONTINUED

Applications:

1. **Embedded Systems:** Programs for devices like control systems and peripherals are often written in C, Java, or assembly for efficiency and hardware interaction.
2. **Game Development:** Assembly language optimizes performance in game machines with strict memory and speed requirements.
3. **Early Programming:** Early programs were written in assembly due to constraints, but higher-level languages like C and Java are now more common.
4. **Optimization:** Assembly language is used to optimize critical sections of applications for speed and hardware access despite its maintenance complexity.

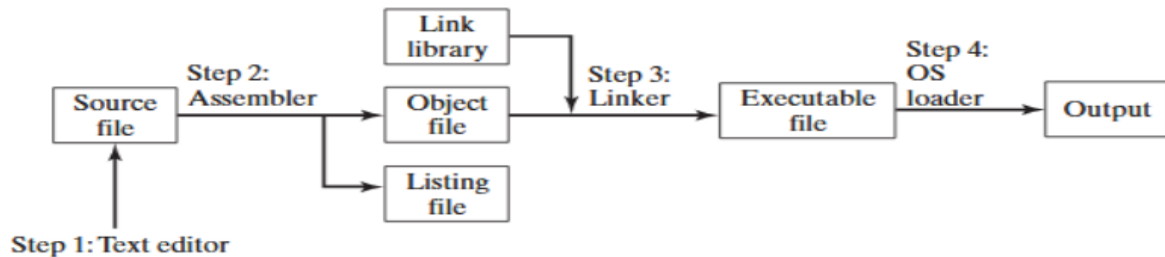
Type of Application	High-Level Languages	Assembly Language
Commercial or scientific application, written for single platform, medium to large size.	Formal structures make it easy to organize and maintain large sections of code.	Minimal formal structure, so one must be imposed by programmers who have varying levels of experience. This leads to difficulties maintaining existing code.
Hardware device driver.	The language may not provide for direct hardware access. Even if it does, awkward coding techniques may be required, resulting in maintenance difficulties.	Hardware access is straightforward and simple. Easy to maintain when programs are short and well documented.
Commercial or scientific application written for multiple platforms (different operating systems).	Usually portable. The source code can be recompiled on each target operating system with minimal changes.	Must be recoded separately for each platform, using an assembler with a different syntax. Difficult to maintain.
Embedded systems and computer games requiring direct hardware access.	May produce large executable files that exceed the memory capacity of the device.	Ideal, because the executable code is small and runs quickly.

Portability: Assembly language is not portable because it is designed for specific processor families, like Motorola 68x00, x86, with different languages for each family, unlike high-level languages like C++ and Java which can run on various systems.

Assembly-Link Execute Cycle: An assembly language source program must be assembled into an object file and then linked to produce an executable file before it can be run on a computer.

1. **Create Source File:** A programmer uses a text editor to create an ASCII text file, the source file.

2. **Assemble:** The assembler translates the source file into an object file (machine code) and optionally a listing file; errors require returning to Step 1.
3. **Link:** The linker combines the object file with necessary library procedures to produce an executable file.
4. **Execute:** The operating system loader reads the executable file into memory and starts the program.



Assembly Language for x86 Processors: Intel stands for "Integrated Electronics," and AMD stands for "Advanced Micro Devices." Assembly Language for x86 Processors focuses on programming microprocessors compatible with Intel IA-32 (Intel Architecture, 32-bit) and AMD x86 processors running under Microsoft Windows. Assembly language is the closest high-level representation of native machine language.

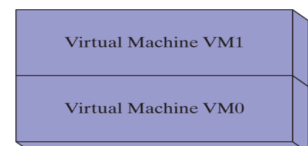
VIRTUAL MACHINES

Virtual Machine (VM) is a software program that emulates the functions of some other physical or virtual computer.

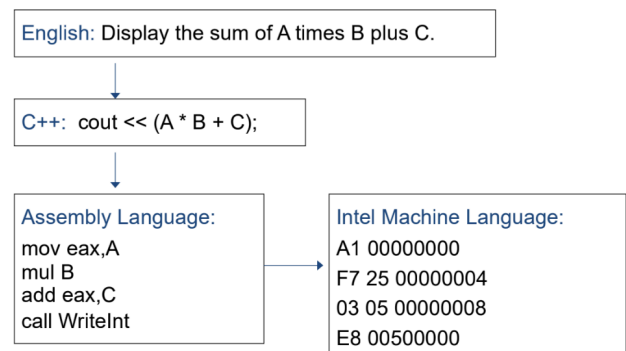
Virtual Machine Concept: Writing in L0 (computer native language) is complex due to its numeric detail, so L1 (user-friendly language) is constructed for easier programming through interpretation or translation. The virtual machine VM1, as we will call it, can execute commands written in language L1. The virtual machine VM0 can execute commands written in language L0:

Interpretation: The L0 program interprets and executes L1 instructions one by one.

Translation: The entire L1 program is translated into L0, which then runs directly on the hardware.



The concept can be repeated across multiple virtual machine levels (VM1, VM2, etc.). Java is based on this concept: Java programs are translated into Java byte code and executed by the Java Virtual Machine (JVM). JVM is platform-dependent which means there are different implementations of JVM on different OS.



Specific Machine Levels: In actual computers, Level 1 represents the digital logic hardware, while Level 2, known as the Instruction Set Architecture (ISA), is where users can write programs in binary machine language.

Level 1: Digital Logic:

- This is the CPU level, constructed from digital logic gates.
- Components: System bus, memory, and digital circuits, implemented using bipolar transistors.

Level 2: Instruction Set Architecture (ISA):

- Processor has a built-in instruction set for basic operations (e.g., move, add, multiply).
- These instructions, known as machine language, are executed by the hardware or a microprogram within the chip.
- This level involves the computer's attributes as seen by the assembly programmer, such as: Programmable storage, Data types and structures, Instruction set, formats, and addressing modes
- Executed by Level 1 (Digital Logic).

Level 3: Assembly Language:

- Uses mnemonics (e.g., ADD, SUB, MOV) that map directly to machine language instructions in a one-to-one correspondence.
- Programs are fully assembled into Instruction Set Architecture (ISA) machine language (Level 2) before execution.
- Instructions may directly match the hardware or be translated by a microcode interpreter within the processor during execution.
- Acts as a bridge between machine language and higher-level programming, making large-scale software development more practical.

Level 4: High-Level Languages:

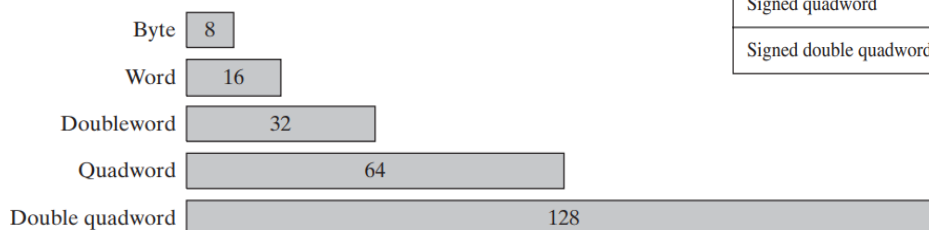
- Includes languages such as C++, Java, Pascal, and Visual Basic.
- Programs are compiled into assembly language (Level 3).
- High-level statements translate into multiple assembly language instructions.
- Compilers automatically convert assembly language code into machine language.

DATA REPRESENTATION

Table 1-7 Ranges and Sizes of Signed Integer Types.

System	Base	Possible Digits
Binary	2	0 1
Octal	8	0 1 2 3 4 5 6 7
Decimal	10	0 1 2 3 4 5 6 7 8 9
Hexadecimal	16	0 1 2 3 4 5 6 7 8 9 A B C D E F

Type	Range	Storage Size in Bits
Signed byte	-2^7 to $+2^7 - 1$	8
Signed word	-2^{15} to $+2^{15} - 1$	16
Signed doubleword	-2^{31} to $+2^{31} - 1$	32
Signed quadword	-2^{63} to $+2^{63} - 1$	64
Signed double quadword	-2^{127} to $+2^{127} - 1$	128



Hexadecimal Addition

Carry	1		
X	6	A	2
Y	4	9	A
S	B	3	C

To create the two's complement of a hexadecimal integer, reverse all bits and add 1. An easy way to reverse the bits of a hexadecimal digit is to subtract the digit from 15. Here are examples of hexadecimal integers converted to their two's complements:

6A3D --> 95C2 + 1 --> 95C3
95C3 --> 6A3C + 1 --> 6A3D

BINARY TO DECIMAL (& REVERSE)

$(49)_{10} \rightarrow (00110001)_2$
 ns 64 32 16 8 4 2 1
 0 0 1 1 0 0 0 1

BINARY TO HEXADECIMAL (& REVERSE)

$(00110001)_2 \rightarrow (31)_{16}$
 8 4 2 1 8 4 2 1
 0 0 1 1 0 0 0 1
 3 1

BINARY TO OCTAL (& REVERSE)

$(00110001)_2 \rightarrow (061)_8$
 4 2 1 4 2 1 4 2 1
 0 0 0 1 1 0 0 0 1
 0 6 1

DECIMAL TO HEXADECIMAL (& REVERSE)

$(49)_{10} \rightarrow (31)_{16}$

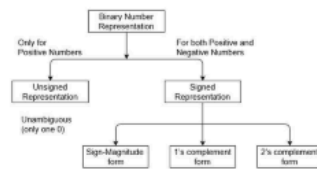
 $(31)_{16} \rightarrow (49)_{10}$

BINARY ADDITION

10101
 $+ 10111$
 101100

BINARY SUBTRACTION

10101
 $- 00111$
 01110



1's COMPLEMENT + 2's COMPLEMENT

11001010
 $00110101 \rightarrow 1's \text{ COMPLEMENT}$
 $+ 1$
 $00110110 \rightarrow 2's \text{ COMPLEMENT}$

SIGNED MAGNITUDE FORM

5 64 32 16 8 4 2 1 } -49
 1 0 1 1 0 0 0 1

BINARY TO GRAY CODE (& REVERSE)

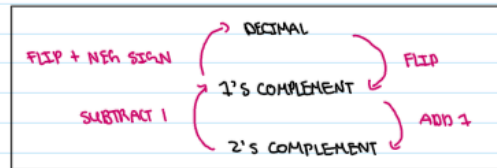
$1 \rightarrow 0 \rightarrow 1 \rightarrow 1 \rightarrow 0$
 BIN TO GRAY

$1 \rightarrow 0 \rightarrow 1 \rightarrow 1 \rightarrow 0$
 GRAY TO BIN

BCD ADDITION

$983 + 187 = 1170$

NOTE :- 1's and 2's complement remain same for positive numbers.



Negative integers use two's-complement representation, using the mathematical principle that the two's complement of an integer is its additive inverse. (If you add a number to its additive inverse, the sum is zero.)

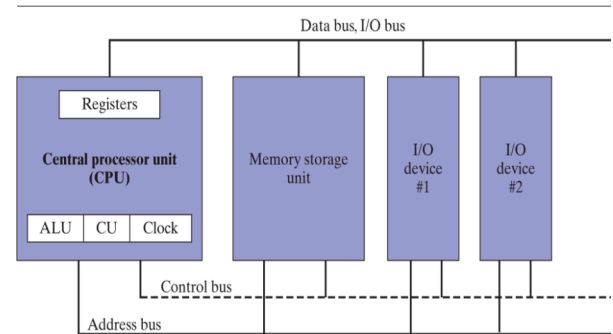
DECIMAL	POSITIVE BINARY	SIGNED MAGNITUDE	1'S COMPLEMENT	2'S COMPLEMENT
21	00010101	00010101	00010101	00010101
-49	00110001	10110001	11001110	11001111

Not signed magnitude
 Its signed binary only here

BASIC MICROCOMPUTER DESIGN

Microcomputer: A computer with a microprocessor (μ P) as its central processing unit (CPU).

Microprocessor (μ P): A digital component with transistors on a single semiconductor integrated circuit (IC).



Components of a Microcomputer:

- **Central Processing Unit (CPU):** Where calculations and logical operations take place
 - **Registers:** Store data and instructions used by the processor.
 - **Clock:** Synchronizes internal CPU operations with other system components.
 - **Control Unit (CU):** Coordinates instruction execution, fetches and decodes instructions.
 - **Arithmetic Logic Unit (ALU):** Performs arithmetic (addition, subtraction) and logical operations (AND, OR, NOT).
- **Memory Storage Unit:** Holds instructions and data during program execution, transfers data between RAM and CPU.

CPU Connections: The CPU is attached to the rest of the computer via pins attached to the CPU socket in the computer's motherboard. Most pins connect to the data bus, the control bus, and the address bus.

Data Processing: Programs must be loaded into the CPU from memory to execute, either individually or in groups.

Buses:

- **Data Bus:** Transfers instructions and data between CPU and memory.
- **I/O Bus:** Transfers data between CPU and system input/output devices.
- **Control Bus:** Synchronizes actions of all system devices with binary signals.
- **Address Bus:** Carries addresses for data transfers between CPU and memory.

Clock: Synchronizes operations involving the CPU and the system bus with a constant rate of pulses.

- **Machine Cycle:** Basic unit of time for machine instructions; duration is one clock cycle.
- **Clock Cycle:** Time required for one complete clock pulse; measured from one falling edge to the next.
- **Duration Calculation:** Reciprocal of the clock's speed (oscillations per second). For example, a 1 GHz clock has a cycle duration of 1 nanosecond.
- **Instruction Execution:** Requires at least one clock cycle; some instructions need more than 50 cycles (e.g., multiply instruction on the 8088 processor).
- **Wait States:** Empty clock cycles that occur during memory access due to speed differences between the CPU, system bus, and memory circuits.

INSTRUCTION EXECUTION CYCLE

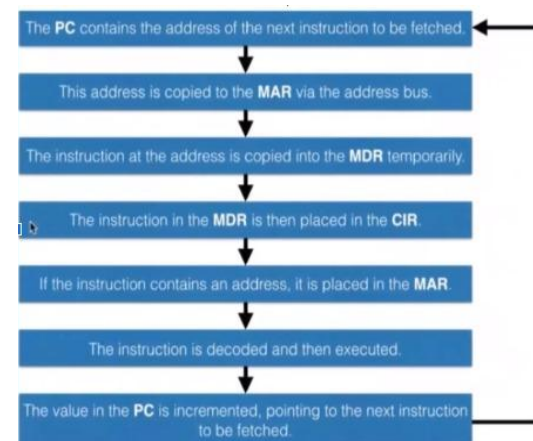
Instruction Execution Cycle: The CPU executes a machine instruction through a predefined sequence of steps, known as the instruction execution cycle, guided by the instruction pointer (IP) register.

1. **Fetch:** The CPU fetches the instruction from the instruction queue in memory and increments the instruction pointer.
 2. **Decode:** The CPU decodes the instruction by analyzing its binary bit pattern, which may indicate operands (input values).
 3. **Fetch Operands:** If operands are involved, the CPU retrieves them from registers or memory, which may include address calculations.
 4. **Execute:** The CPU performs the instruction using the fetched operands and updates status flags such as Zero, Carry, and Overflow.
 5. **Store Result:** If there is an output operand, the CPU stores the result of the execution in the operand.
- **Simplified Steps:** The instruction execution process is commonly summarized as Fetch, Decode, and Execute.
 - **Operand Definition:** An operand is a value used as input or output for an operation, e.g., in $Z = X + Y$, X and Y are input operands and Z is the output operand.
 - **Addressing and Data Transfer:** To read instructions, an address is placed on the address bus; the memory controller then places the code on the data bus, making it available in the code cache.
 - **Instruction Pointer and Decoding:** The instruction pointer determines the next instruction to execute; the instruction decoder analyzes it and sends signals to the control unit, coordinating with the ALU and floating-point unit.
 - **Control Bus Role:** The control bus uses the system clock to synchronize data transfer between different CPU components.

Reading from Memory: Computers read memory more slowly than accessing internal registers, involving four steps, each requiring a clock cycle.

1. Place the address of the value on the address bus.
2. Assert the processor's RD (read) pin.
3. Wait one clock cycle for the memory to respond. (copied to data bus)
4. Copy the data from the data bus into the destination operand.

Clock Speed: Each clock cycle is brief, with speeds like 1.2 GHz making each cycle last 1 nanosecond, but this is still slower compared to internal register access.



Cache Memory: To reduce memory access time (speed bottleneck) by storing recently used instructions and data in high-speed memory. Cache is used because programs frequently access the same memory and instructions repeatedly. The CPU loads upcoming instructions into cache to preemptively manage future needs, improving performance.

X86 Cache Types:

Level-1 Cache (L1): Primary cache stored directly on the CPU, offering the fastest access.

Level-2 Cache (L2): Secondary cache, slightly slower, attached to the CPU via a high-speed data bus.

Cache Operations: The CPU preloads the next set of instructions into cache, anticipating they will be needed soon, and if there's a loop, those instructions remain in cache for quicker access.

- **Cache Hit:** When the CPU finds the data in the cache.
- **Cache Miss:** When the CPU cannot find the data in the cache, requiring access to slower memory.

Cache vs. Conventional RAM

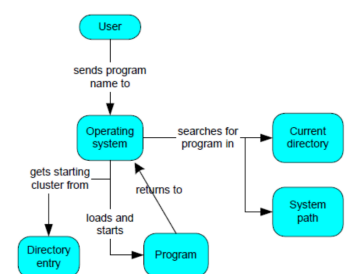
Static RAM (SRAM): Used in cache memory, faster and does not require constant refreshing but is more expensive.

Dynamic RAM (DRAM): Used in conventional RAM, slower, cheaper, and requires constant refreshing to maintain its contents.

LOADING AND EXECUTING A PROGRAM:

Loading and Executing a Program:

1. **Filename Search:** The OS searches for the program's filename in the current directory and then in a list of directories (paths); issues an error if not found.
2. **File Retrieval:** If found, the OS retrieves file information, including size and location.
3. **Memory Allocation:** The OS finds an available memory location, loads the program file into memory, and updates the descriptor table with size and location details.
4. **Pointer Adjustment:** The OS adjusts pointers within the program to reflect correct addresses.
5. **Execution Start:** The OS starts execution at the program's entry point (first machine instruction), turning it into a process.
6. **Process Management:** The OS assigns a process ID, tracks execution, and manages system resources like memory and I/O devices.
7. **Process Termination:** When the process ends, it is removed from memory.



Modes of Operation for x86 Processors

Real-Address Mode: Original mode (8086) allowing direct access to up to 1 MB of memory.

- Characteristics: Programs access any part of main memory; used by MS-DOS.
- Limitations: Can cause system crashes due to lack of memory protection.

❖ A program can access up to six segments at any time

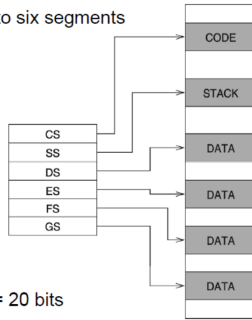
- ❖ Code segment
- ❖ Stack segment
- ❖ Data segment
- ❖ Extra segments (up to 3)

❖ Each segment is 64 KB

❖ Logical address

- ❖ Segment = 16 bits
- ❖ Offset = 16 bits

❖ Linear (physical) address = 20 bits



Linear address = Segment × 10 (hex) + Offset

Example:

segment = A1F0 (hex)

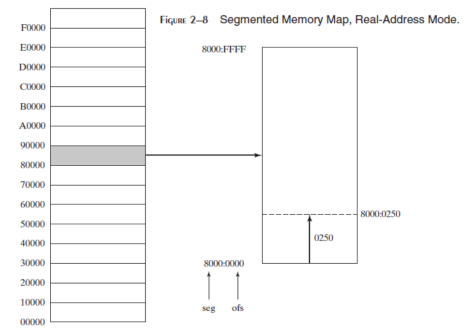
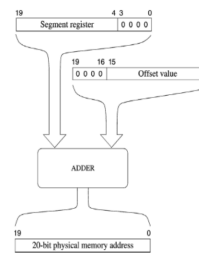
offset = 04C0 (hex)

logical address = A1F0:04C0 (hex)

what is the linear address?

Solution:

A1F00 (add 0 to segment in hex)
 + 04C0 (offset in hex)
 A23C0 (20-bit linear address in hex)



Protected Mode: Introduced with the 80386 processor; native mode of modern x86 CPUs.

- Characteristics: Addresses up to 4 GB of memory; OS assigns and protects memory segments; supports multitasking and virtual memory.
- Usage: Used by modern operating systems like Windows NT, 2000, XP, and Linux.

Virtual-8086 Mode: A sub-mode of protected mode that emulates a real-address environment.

- Characteristics: Provides a virtual 8086 machine with 1 MB address space for running legacy programs; crashes in virtual-8086 mode do not affect other processes.
- Usage: Allows multiple virtual-8086 sessions in modern OS like Windows XP.

System Management Mode (SMM): Used for system management tasks such as power management and system security.

- Characteristics: Manages functions like thermal control, power management, and system event handling, safety functions.
- Usage: Custom implementations by manufacturers for system safety and performance (memory or chipset errors)

Difference:

Protected Mode: Provides memory protection by using virtual addresses managed by the OS, allowing up to 4 GB of addressable space, and separates user and system programs.

Real Mode: Offers direct access to physical memory with no protection, limited to 1 MB, and can lead to potential system conflicts with no isolation of user and system programs.

Although the 8086 processor has a 20-bit address bus, the segment address and offset are not in 20-bit representation. Explain why?

The 8086 processor uses a 16-bit Instruction Pointer and 16-bit segment register, where the segment address is shifted left by 4 bits and is then added to the 16-bit offset to form a 20-bit physical address, allowing access to 1MB of memory

In the 8086 processor, you have two 16-bit values—the segment address and the offset (The IP is a 16-bit register that holds the offset)—but they don't simply "add up" to make a 20-bit address. Instead, the processor uses a clever method to create a 20-bit address.

Here's how it works step-by-step in simple terms:

Segment Address (16-bit): This is like a base address that points to a block of memory. But it needs to be adjusted to point to the correct location in memory. To do this, the processor takes the 16-bit segment address and shifts it left by 4 bits (which is the same as multiplying it by 16). This gives a larger value that can point to memory in chunks of 16 bytes.

Offset (16-bit): The offset is another 16-bit value that tells the processor the exact position within that block (segment) of memory. It tells the processor how many steps to take from the base (segment) to find the exact spot in memory.

Combining Segment and Offset: The shifted segment address (now 20 bits) and the offset are added together. This combination gives a 20-bit physical address, which allows the processor to access up to 1MB of memory.

BASIC EXECUTION ENVIRONMENT

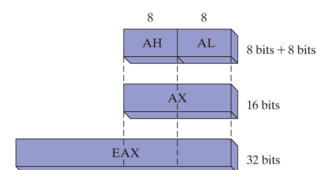
Address Space + Memory Management:

- **Real-Address Mode:** Allows access to 1 MB memory/address space, single-tasking with direct hardware access, used in MS-DOS and Windows 95/98.
- **Protected Mode:** Supports up to 4 GB linear address space per program, multitasking, with memory isolation, used in MS-Windows and Linux.
- **Virtual-8086 Mode:** Each program gets 1 MB memory space, emulates real-address mode within protected mode, used in Windows NT/2000.
- **Extended Physical Addressing:** Allows up to 64 GB physical memory in protected mode starting from P6 processors.

Registers:

- **General-Purpose Registers (8):** primarily used for arithmetic and data movement
 - **EAX:** Extended Accumulator, used in arithmetic, logic, and control.
 - **EBX:** Extended Base, often used as an address register.
 - **ECX:** Extended Counter, used as a loop counter.
 - **EDX:** Extended Data, used in multiplication and division.
 - **AX, BX, CX, DX:** 16-bit versions of EAX, EBX, ECX, and EDX, respectively. Each has high (AH, BH, CH, DH) and low (AL, BL, CL, DL) 8-bit parts.
- **Pointer and Index Registers:** contain pointers and the offsets for data and instructions (stack and string operations)
 - **ESP:** Extended Stack Pointer, points to the top of the stack.
 - **EBP:** Extended Base Pointer, references function parameters and local variables.
 - **ESI:** Extended Source Index, points to the source in string operations.
 - **EDI:** Extended Destination Index, points to the destination in string operations.
- **Segment Registers (6):** hold pointers to segment descriptor tables and hold base addresses for code, data, extra, and stack segments in a program.

Types:



- **Code Segment:** Base address for executable instructions.
- **Data Segment:** Base address for variables and program data.
- **Extra Segment:** Additional data segment, often for shared data.
- **Stack Segment:** Base address for the stack, stores local variables and return addresses, including interrupt and subroutine return addresses.
- **Far Segment:** Used for accessing thread-local storage (TLS) in certain operating systems.
- **Global Segment:** Often points to system-specific data, such as process or CPU-related structures in 64-bit systems.

Uses:

- **Code Segment:** For executable instructions.
- **Data Segment:** For program variables.
- **Extra Segment:** For additional data or shared data.
- **Stack Segment:** For stack operations, including interrupts and subroutine returns.
- **Instruction Pointer (EIP):** Contains the address of the next instruction to execute.
- **Flags Register (EFLAGS):** Controls CPU operations and reflects outcomes of operations using binary bits.
 - **Control Flags:** Determine how instructions are executed.
 - **Direction Flag (DF):** Determines data transfer direction (1 = up; 0 = down).
 - **Interrupt Flag (IF):** Controls whether interrupts are enabled, allowing devices to interrupt the CPU.
 - **Trap Flag (TF):** Halts the CPU after each instruction for debugging purposes.
 - **Status Flags:** Report the outcome of operations.
 - **Carry Flag (CF):** Set if an unsigned arithmetic result exceeds the destination size.
 - **Overflow Flag (OF):** Set if a signed arithmetic result overflows or underflows.
 - **Sign Flag (SF):** Set if the result is negative.
 - **Zero Flag (ZF):** Set if the result is zero.
 - **Auxiliary Carry Flag (AC):** Set if there's a carry from bit 3 to bit 4 in an 8-bit operand.
 - **Parity Flag (PF):** Set if the least-significant byte has an even number of 1 bits; used for error checking.
- **Specialized Registers:**
 - **MMX Registers:** Eight 64-bit registers for SIMD (Single-Instruction, Multiple-Data) operations in multimedia applications, operate in parallel on the data values contained in MMX registers
 - **XMM Registers:** Eight 128-bit registers used by streaming SIMD extensions to the instruction set
 - **Floating-Point Unit (FPU):** Contains eight floating-point registers (ST(0) to ST(7)) for high-speed floating-point arithmetic.

Memory Segmentation:

- **Segments:** Blocks of memory identified by segment numbers, starting with 0.
- **Segment:Offset:** Format to specify memory locations within a segment (e.g., A4FB:4872h)

ASSEMBLY LANGUAGE FUNDAMENTALS (CODING)

```
1: .data                ; this is the data area
2: sum DWORD 0          ; create a variable named
sum
3:
4: .code                ; this is the code area
5: main PROC
6:   mov eax,5           ; move 5 to the eax
register
7:   add eax,6           ; add 6 to the eax register
8:   mov sum,eax
9:
10:  INVOKE ExitProcess,0 ; end the program
11: main ENDP
```

The following example helps to show the difference between directives and instructions. The **DWORD** directive tells the assembler to reserve space in the program for a doubleword variable. The **MOV** instruction, on the other hand, executes at runtime, copying the contents of **myVar** to the **EAX** register:

```
myVar  DWORD 26
mov     eax,myVar
```

INTEGER LITERALS [{ + | - }] digits [radix]

h	hexadecimal	r	encoded real
q/o	octal	t	decimal (alternate)
d	decimal	y	binary (alternate)
b	binary		

A hexadecimal literal beginning with a letter must have a leading zero to prevent the assembler from interpreting it as an identifier. Such is the

Operator	Name	Precedence Level
()	Parentheses	1
+, -	Unary plus, minus	2
*, /	Multiply, divide	3
MOD	Modulus	3
+, -	Add, subtract	4

[label:] mnemonic [operands] [;comment]

An **instruction** is an executable statement translated into machine code by the assembler, containing **operands** (up to three) which can be registers, memory, or expressions. In assembly, with multiple operands (**IMUL** instruction), the first is the destination (modified by the instruction), the second is the source, and the third is typically an immediate value (used for calculations) - `imul eax,ebx,5`. A **directive** in assembly language is an instruction to the assembler that controls how the program is assembled, without generating machine code. **.code** and **.data** directives, are called **segments**. A **segment** is a designated storage area that holds either program code, program variables (data), or the stack. An **identifier** is a programmer-chosen name. A **label** is an identifier that marks the memory location of data (data label) or an instruction (code label). **Data labels** reference variables (e.g., `count DWORD 100`), while **code labels** serve as jump targets (e.g., `target: mov ax,bx ... jmp target`). The **NOP (No Operation)** instruction, taking 1 byte, performs no work but is used to align code for efficient memory access, especially on doubleword boundaries.

Mnemonic	Description
MOV	Move (assign) one value to another
ADD	Add two values
SUB	Subtract one value from another
MUL	Multiply two values
JMP	Jump to a new location
CALL	Call a procedure

Example	Operand Type
96	Integer literal
2 + 4	Integer expression
eax	Register
count	Memory

```
00000000 66 8B C3      mov ax,bx
00000003 90              nop
instruction
00000004 8B D1      mov edx,ecx
```

; align next

- Main PROC (program entry point, starts procedure)
- Main ENDP (ends procedure)
- END main (end program)

Following are examples of valid decimal reals:

```
2.
+3.0
-44.2E+05
26.E5
```

Defining Data:

[name] directive initializer [,initializer] . . .

Table 3-2 Intrinsic Data Types.

Type	Usage
BYTE	8-bit unsigned integer. B stands for byte
SBYTE	8-bit signed integer. S stands for signed
WORD	16-bit unsigned integer
SWORD	16-bit signed integer
DWORD	32-bit unsigned integer. D stands for double
SDWORD	32-bit signed integer. SD stands for signed double
FWORD	48-bit integer (Far pointer in protected mode)
QWORD	64-bit integer. Q stands for quad
TBYTE	80-bit (10-byte) integer. T stands for Ten-byte
REAL4	32-bit (4-byte) IEEE short real
REAL8	64-bit (8-byte) IEEE long real
REAL10	80-bit (10-byte) IEEE extended real

```
BYTE 20 DUP(0)           ; 20 bytes, all equal to zero
BYTE 20 DUP(?)           ; 20 bytes, uninitialized
BYTE 4 DUP("STACK")      ; 20 bytes: "STACKSTACKSTACKSTACK"
```

```
greeting1 BYTE "Good afternoon",0
greeting2 BYTE 'Good night',0
```

```
greeting1 BYTE "Welcome to the Encryption Demo program "
            BYTE "created by Kip Irvine.",0dh,0ah
            BYTE "If you wish to modify this program, please "
            BYTE "send me a copy.",0dh,0ah,0
```

The hexadecimal codes 0Dh and 0Ah are alternately called CR/LF (carriage-return line-feed) or *end-of-line characters*. When written to standard output, they move the cursor to the left column of the line following the current line.

The line continuation character (\) concatenates two source code lines into a single statement. It must be the last character on the line. The following statements are equivalent:

```
greeting1 BYTE "Welcome to the Encryption Demo program "
and
greeting1 \
BYTE "Welcome to the Encryption Demo program "
```

In **little-endian order**, x86 processors store the least significant byte first (e.g., 12345678h as 78h 56h 34h 12h), while in **big-endian order**, the most significant byte is stored first (e.g., 12345678h as 12h 34h 56h 78h).

Table 3-3 Legacy Data Directives.

Directive	Usage
DB	8-bit integer
DW	16-bit integer
DD	32-bit integer or real
DQ	64-bit integer or real
DT	define 80-bit (10-byte) integer

```
value1 BYTE 'A'           ; character constant
value2 BYTE 0              ; smallest unsigned byte
value3 BYTE 255            ; largest unsigned byte
value4 SBYTE -128          ; smallest signed byte
value5 SBYTE +127          ; largest signed byte
value6 BYTE ?              ; uninitialized byte
```

```
word1 WORD 65535           ; largest unsigned value
word2 SWORD -32768         ; smallest signed value
```

If multiple initializers are used in the same data definition, its label refers only to the offset of the first initializer. In the following example, assume

```
list BYTE 10,20,30,40
```

Figure 3-11 shows **list** as a sequence of bytes, each with its own offset.

Figure 3-11 Memory layout of a byte sequence.

Offset	Value
0000:	10
0001:	20
0002:	30
0003:	40

```
BYTE 20 DUP(0)           ; 20 bytes, all equal to zero
BYTE 20 DUP(?)           ; 20 bytes, uninitialized
BYTE 4 DUP("STACK")      ; 20 bytes:
"STACKSTACKSTACKSTACK"
```

Offset	Value	Offset	Value
0000:	1	0000:	1
0002:	2	0004:	2
0004:	3	0008:	3
0006:	4	000C:	4
0008:	5	0010:	5

Symbolic constants in assembly language are used to represent integer expressions or text without reserving storage. The **equal-sign directive** associates a symbol with an integer expression (e.g., COUNT = 500), which can be used in place of the literal value, simplifying updates. The **EQU directive** assigns an integer expression, symbol, or text value (e.g., PI EQU <3.1416>), but cannot be redefined. The **TEXT EQU directive** creates text macros (e.g., continueMsg TEXT EQU <"Do you wish to continue (Y/N)?">) and allows for redefinition. Example: M1 WORD matrix1 where matrix1 EQU 10 * 10 results in M1 WORD 100.

	Symbol	Variable
Uses storage?	No	Yes
Value changes at runtime?	No	Yes

Differentiate between Statement & Instruction.

Statement: A line of code in a high-level language that can include variable declarations, loops, and instructions (e.g., int a = 5;).

Instruction: A low-level command in assembly or machine language that tells the CPU to perform a specific operation (e.g., MOV ax, bx).

Rather than calculating the length of a string manually, let the assembler do it:

```
myString BYTE "This is a long string, containing"  
          BYTE "any number of characters"  
myString_len = ($ - myString)
```

Arrays of Words and DoubleWords When calculating the number of elements in an array containing values other than bytes, you should always divide the total array size (in bytes) by the size of the individual array elements. The following code, for example, divides the address range by 2 because each word in the array occupies 2 bytes (16 bits):

```
list WORD 1000h,2000h,3000h,4000h  
ListSize = ($ - list) / 2
```

DATA TRANSFERS, ADDRESSING, & ARITHMETIC

```
[label:] mnemonic [operands][ ; comment ]
```

Instructions can have zero, one, two, or three operands. Here, we omit the label and comment fields for clarity:

```
mnemonic          MOV reg, reg
mnemonic [destination] MOV mem, reg
mnemonic [destination], [source] MOV reg, mem
mnemonic [destination], [source-1], [source-2] MOV mem, imm
mnemonic [destination], [source-1], [source-2] MOV reg, imm
```

There are three basic types of operands:

- Immediate—uses a numeric literal expression
- Register—uses a named register in the CPU
- Memory—references a memory location

MOVZX Instruction

The MOVZX instruction (*move with zero-extend*) copies the contents of a source operand into a destination operand and zero-extends the value to 16 or 32 bits. This instruction is only used with unsigned integers. There are three variants:

```
MOVZX reg32, reg/mem8
MOVZX reg32, reg/mem16
MOVZX reg16, reg/mem8
```

MOVSX Instruction

The MOVSX instruction (*move with sign-extend*) copies the contents of a source operand into a destination operand and sign-extends the value to 16 or 32 bits. This instruction is only used with signed integers. There are three variants:

```
MOVSX reg32, reg/mem8
MOVSX reg32, reg/mem16
MOVSX reg16, reg/mem8
```

An operand is sign-extended by taking the smaller operand's highest bit and repeating (replicating) the bit throughout the extended bits in the destination operand. The following example

4.1.6 LAHF and SAHF Instructions

The LAHF (load status flags into AH) instruction copies the low byte of the EFLAGS register into AH. The following flags are copied: Sign, Zero, Auxiliary Carry, Parity, and Carry. Using this instruction, you can easily save a copy of the flags in a variable for safekeeping:

```
.data
saveflags BYTE ?
.code
lahf                ; load flags into AH
mov saveflags, ah   ; save them in a variable
```

The SAHF (store AH into status flags) instruction copies AH into the low byte of the EFLAGS (or RFLAGS) register. For example, you can retrieve the values of flags saved earlier in a variable:

```
mov ah, saveflags   ; load saved flags into AH
sahf                ; copy into Flags register
```

4.1.7 XCHG Instruction

The XCHG (exchange data) instruction exchanges the contents of two operands. There are three variants:

```
XCHG reg, reg
XCHG reg, mem
XCHG mem, reg
```

The rules for operands in the XCHG instruction are the same as those for the MOV instruction (Section 4.1.4), except that XCHG does not accept immediate operands. In array sorting applications, XCHG provides a simple way to exchange two array elements. Here are a few

4.2.4 NEG Instruction

The NEG (negate) instruction reverses the sign of a number by converting the number to its two's complement. The following operands are permitted:

```
NEG reg
NEG mem
```

```
.data
var1 BYTE 10h
.code
mov al, var1           ; AL = 10h
mov al, [var1]         ; AL = 10h
```

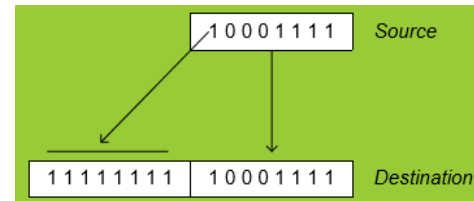
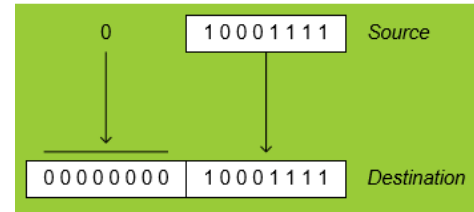


Table 4-1 Instruction Operand Notation, 32-Bit Mode.

Operand	Description
reg8	8-bit general-purpose register: AH, AL, BH, BL, CH, CL, DH, DL
reg16	16-bit general-purpose register: AX, BX, CX, DX, SI, DI, SP, BP
reg32	32-bit general-purpose register: EAX, EBX, ECX, EDX, ESI, EDI, ESP, EBP
reg	Any general-purpose register
sreg	16-bit segment register: CS, DS, SS, ES, FS, GS
imm	8-, 16-, or 32-bit immediate value
imm8	8-bit immediate byte value
imm16	16-bit immediate word value
imm32	32-bit immediate doubleword value
reg/mem8	8-bit operand, which can be an 8-bit general register or memory byte
reg/mem16	16-bit operand, which can be a 16-bit general register or memory word
reg/mem32	32-bit operand, which can be a 32-bit general register or memory doubleword
mem	An 8-, 16-, or 32-bit memory operand

Overlapping Values

The following code example shows how the same 32-bit register can be modified using differently sized data. When **oneWord** is moved to AX, it overwrites the existing value of AL. When **oneDword** is moved to EAX, it overwrites AX. Finally, when 0 is moved to AX, it overwrites the lower half of EAX.

```
.data
oneByte BYTE 78h
oneWord WORD 1234h
oneDword DWORD 12345678h
.code
mov eax, 0           ; EAX = 00000000h
mov al, oneByte      ; EAX = 00000078h
mov ax, oneWord      ; EAX = 00001234h
mov eax, oneDword    ; EAX = 12345678h
mov ax, 0            ; EAX = 12340000h
```


4.2.1 INC and DEC Instructions

The INC (increment) and DEC (decrement) instructions, respectively, add 1 and subtract 1 from a register or memory operand. The syntax is

```
INC reg/mem
DEC reg/mem
```

Following are some examples:

```
.data
myWord WORD 1000h
.code
inc myWord           ; myWord = 1001h
mov bx,myWord
dec bx               ; BX = 1000h
```

The Overflow, Sign, Zero, Auxiliary Carry, and Parity flags are changed according to the value of the destination operand. The INC and DEC instructions do not affect the Carry flag (which is something of a surprise).

The Addition Test

There is a very easy way to tell whether signed overflow has occurred when adding two operands. Overflow occurs when:

- Adding two positive operands generates a negative sum
- Adding two negative operands generates a positive sum

Overflow never occurs when the signs of two addition operands are different.

Direct-Offset Operands

- **WORD Arrays:** In an array of 16-bit words, the offset of each array element is 2 bytes beyond the previous one.
- That's why 2 is added as offset in Array (for each next element of an array)

```
.data
arrayW WORD 100h,200h,300h
.code
mov ax,arrayW           ; AX = 100h
mov ax,[arrayW+2]       ; AX = 200h
```

In **direct addressing**, the memory address of the data is provided directly, either using the label of the variable or a constant offset.

- **DWORD Arrays:** In an array of 32-bit words, the offset of each array element is 4 bytes beyond the previous one.
- That's why 4 is added as offset in Array (for each next element of an array)

```
.data
arrayD DWORD 10000h,20000h
.code
mov eax,arrayD          ; EAX = 10000h
mov eax,[arrayD+4]      ; EAX = 20000h
```

A displacement added to a variable name produces an effective address (EA) by combining the base address with the constant, allowing access to specific memory locations.

- The **OFFSET** operator returns the distance of a variable from the beginning of its enclosing -segment.
- The **PTR** operator lets you override an operand's default size.
- The **TYPE** operator returns the size (in bytes) of an operand or of each element in an array.
- The **LENGTHOF** operator returns the number of elements in an array.
- The **SIZEOF** operator returns the number of bytes used by an array initializer.

4.3.3 PTR Operator

You can use the PTR operator to override the declared size of an operand. This is only necessary when you're trying to access the operand using a size attribute that is different from the one assumed by the assembler.

Suppose, for example, that you would like to move the lower 16 bits of a doubleword variable named **myDouble** into AX. The assembler will not permit the following move because the operand sizes do not match:

```
.data
myDouble DWORD 12345678h
.code
mov ax,myDouble           ; error
```

But the WORD PTR operator makes it possible to move the low-order word (5678h) to AX:

```
mov ax,WORD PTR myDouble
```

Why wasn't 1234h moved into AX? x86 processors use the *little endian* storage format (Section 3.4.9), in which the low-order byte is stored at the variable's starting address. In Figure 4-7, the memory layout of **myDouble** is shown three ways: first as a doubleword, then as two words (5678h, 1234h), and finally as four bytes (78h, 56h, 34h, 12h).

We can access memory in any of these three ways, independent of the way a variable was defined. For example, if **myDouble** begins at offset 0000, the 16-bit value stored at that address is 5678h. We could also retrieve 1234h, the word at location **myDouble+2**, using the following statement:

```
mov ax,WORD PTR [myDouble+2] ; 1234h
```

OFFSET Examples

In the next example, we declare three different types of variables:

```
.data
bVal BYTE ?
wVal WORD ?
dVal DWORD ?
dVal2 DWORD ?
```

In **indirect addressing**, you use a pointer register (e.g., ESI, EDI, etc.) to store the address of a variable, then access the data through that pointer. The **OFFSET** keyword is often used to get the memory address of a label.

If **bVal** were located at offset 00404000 (hexadecimal), the **OFFSET** operator would return the following values:

```
mov esi,OFFSET bVal      ; ESI = 00404000h
mov esi,OFFSET wVal      ; ESI = 00404001h
mov esi,OFFSET dVal      ; ESI = 00404003h
mov esi,OFFSET dVal2     ; ESI = 00404007h
```

OFFSET can also be applied to a direct-offset operand. Suppose **myArray** contains five 16-bit words. The following **MOV** instruction obtains the offset of **myArray**, adds 4, and moves the resulting address to ESI. We can say that ESI points to the third integer in the array:

```
.data
myArray WORD 1,2,3,4,5
.code
mov esi,OFFSET myArray + 4
```

You can initialize a doubleword variable with the offset of another variable, effectively creating a pointer. In the following example, **pArray** points to the beginning of **bigArray**:

```
.data
bigArray DWORD 500 DUP(?)
pArray DWORD bigArray
```

The following statement loads the pointer's value into ESI, so the register can point to the beginning of the array:

```
mov esi,pArray
```

LABEL Directive

- The **LABEL** directive assigns an alternate label name and type to an existing storage location. **LABEL** does not allocate any storage of its own.

- A common use of **LABEL** is to provide an alternative name and size attribute for the variable declared next in the data segment.

```
.data
val16 LABEL WORD
val32 DWORD 12345678h
.code
mov ax,val16           ; AX = 5678h
mov dx,[val16+2]       ; DX = 1234h
```

```
.data
LongValue LABEL DWORD
val1 WORD 5678h
val2 WORD 1234h
.code
mov eax,LongValue ; EAX = 12345678h
```

4.3.4 TYPE Operator

The TYPE operator returns the size, in bytes, of a single element of a variable. For example, the TYPE of a byte equals 1, the TYPE of a word equals 2, the TYPE of a doubleword is 4, and the TYPE of a quadword is 8. Here are examples of each:

4.3.5 LENGTHOF Operator

The LENGTHOF operator counts the number of elements in an array, defined by the values appearing on the same line as its label. We will use the following data as an example:

```
.data
byte1    BYTE    10,20,30
array1   WORD    30 DUP(?) ,0,0
array2   WORD    5 DUP(3 DUP(?))
array3   DWORD   1,2,3,4
digitStr BYTE    "12345678",0
```

When nested DUP operators are used in an array definition, LENGTHOF returns the product of the two counters. The following table lists the values returned by each LENGTHOF expression:

Expression	Value
LENGTHOF byte1	3
LENGTHOF array1	30 ÷ 2
LENGTHOF array2	5 * 3
LENGTHOF array3	4
LENGTHOF digitStr	9

If you declare an array that spans multiple program lines, LENGTHOF only regards the data from the first line as part of the array. Given the following data, LENGTHOF myArray would return the value 5:

```
myArray BYTE 10,20,30,40,50
        BYTE 60,70,80,90,100
```

4.3.6 SIZEOF Operator

The SIZEOF operator returns a value that is equivalent to multiplying LENGTHOF by TYPE. In the following example, intArray has TYPE = 2 and LENGTHOF = 32. Therefore, SIZEOF intArray equals 64:

```
.data
intArray WORD 32 DUP(0)
.code
mov  eax,SIZEOF intArray      ; EAX = 64
```

Immediate Mode: ADD A, #10 - Adds the number 10 to the current value in register A, useful for quick arithmetic adjustments.

Direct Mode: STO 3000H, B - Stores the content of register B into the memory address 3000H, used for updating global variables.

Indirect Mode: MOV A, [DI] - Moves data from the location pointed by DI register into A, used for accessing dynamically changing data.

Indexed Mode: MOV C, A[SI] - Moves data from an array A at an index SI into register C, used for array manipulation.

Relative Mode: JPE +20 - Jumps 20 bytes ahead in the program if the zero flag is set, used in conditional branching.

INDIRECT ADDRESSING

- Direct addressing is rarely used for array processing because it is impractical to use constant offsets to address more than a few array elements.
- An indirect operand holds the address of a variable, usually an array or string. It can be dereferenced (just like a pointer).

```
.data
vall BYTE 10h,20h,30h
.code
mov esi,OFFSET vall
mov al,[esi]           ; dereference ESI (AL = 10h)

inc esi
mov al,[esi]           ; AL = 20h

inc esi
mov al,[esi]           ; AL = 30h
```

INDIRECT ADDRESSING

- The size of an operand may not be evident from the context of an instruction.

```
inc [esi] ; error: operand must have size
```

- Because the assembler does not know whether ESI points to a byte, word, doubleword, or some other size. The PTR operator confirms the operand size:

```
inc BYTE PTR [esi]
```

Arrays

- Indirect operands are ideal tools for stepping through arrays.

```
.data
arrayB BYTE 10h,20h,30h
.code
mov esi,OFFSET arrayB
mov al,[esi]           ; AL = 10h

inc esi
mov al,[esi]           ; AL = 20h

inc esi
mov al,[esi]           ; AL = 30h
```

Indexed Operands

- An indexed operand adds a constant to a register to generate an effective address. There are two notational forms:

- [label + reg] label[reg]

```
.data
arrayW WORD 1000h,2000h,3000h
.code
mov esi,0
mov ax,[arrayW + esi]   ; AX = 1000h

mov ax, arrayW[esi]     ; alternate format

add esi,2
add ax,[arrayW + esi]
```

Scale Factors in Indexed Operands

- Indexed operands must take into account the size of each array element when calculating offsets.

- Using an array of doublewords, as in the following example, we multiply the subscript (3) by 4 (the size of a doubleword) to generate the offset of the array element containing 400h:

```
.data
arrayD DWORD 100h, 200h, 300h, 400h
.code
mov esi,3 * TYPE arrayD ; offset of arrayD[3]
mov eax,arrayD[esi]      ; EAX = 400h
```

Scale Factors in Indexed Operands

- The x86 instruction set provides a way for offsets to be calculated, using a scale factor.

- The scale factor is the size of the array component (WORD=2, DWORD=4 or QWORD=8).

```
.data
arrayD DWORD 1,2,3,4
.code
mov esi,3
mov eax,arrayD[esi*4] ; EAX = 4
```

```
mov esi,3 ; subscript
mov eax,arrayD[esi*TYPE arrayD] ; EAX = 4
```

Pointers

You can declare a **pointer** variable that contains the offset of another variable.

```
.data
arrayW WORD 1000h,2000h,3000h
ptrW DWORD arrayW
.code
mov esi,ptrW
mov ax,[esi] ; AX = 1000h
```

Alternate format:

```
ptrW DWORD OFFSET arrayW
```

JMP Instruction

- JMP is an unconditional jump to a label that is usually within the same procedure.
- Syntax: **JMP target**
- Logic: EIP ← target
- Example:

```
top:
.
.
jmp top
```

A jump outside the current procedure must be to a special type of label called a **global label** (see Section 5.5.2.3 for details).

LOOP Instruction

- The LOOP instruction creates a counting loop
- Syntax: **LOOP target**
- Logic:
 - ECX ← ECX – 1
 - if ECX != 0, jump to target
- Implementation:
 - The assembler calculates the distance, in bytes, between the offset of the following instruction and the offset of the target label. It is called the **relative offset**.
 - The relative offset is added to EIP.

LOOP Example

The following loop calculates the sum of the integers 5 + 4 + 3 + 2 + 1:

offset	machine code	source code
00000000	66 B8 0000	mov ax,0
00000004	B9 00000005	mov ecx,5
00000009	66 03 C1	L1: add ax,cx
0000000C	E2 FB	loop L1
0000000E		

When LOOP is assembled, the current location = 0000000E (offset of the next instruction). -5 (FBh) is added to the current location, causing a jump to location 00000009:

```
00000009 ← 0000000E + FB
```

Nested Loop

If you need to code a loop within a loop, you must save the outer loop counter's ECX value. In the following example, the outer loop executes 100 times, and the inner loop 20 times.

```
.data
count DWORD ?
.code
mov ecx,100 ; set outer loop count
L1: mov count,ecx ; save outer loop count
mov ecx,20 ; set inner loop count
L2: .
loop L2 ; repeat the inner loop
mov ecx,count ; restore outer loop count
loop L1 ; repeat the outer loop
```

Summing an Integer Array

The following code calculates the sum of an array of 16-bit integers.

```
.data
intarray WORD 100h,200h,300h,400h
.code
mov edi,OFFSET intarray ; address of intarray
mov ecx,LENGTHOF intarray ; loop counter
mov ax,0 ; zero the accumulator
L1: add ax,[edi] ; add an integer
add edi,TYPE intarray ; point to next integer
loop L1 ; repeat until ECX = 0
```

Copying a String

The following code copies a string from source to target:

```
.data
source BYTE "This is the source string",0
target BYTE SIZEOF source DUP(0)
.code
mov esi,0 ; index register
mov ecx,SIZEOF source ; loop counter
L1: mov al,source[esi] ; get char from source
mov target[esi],al ; store it in the target
inc esi ; move to next character
loop L1 ; repeat for entire string
```

good use of
SIZEOF

Not making notes ahead since it is all coding.